

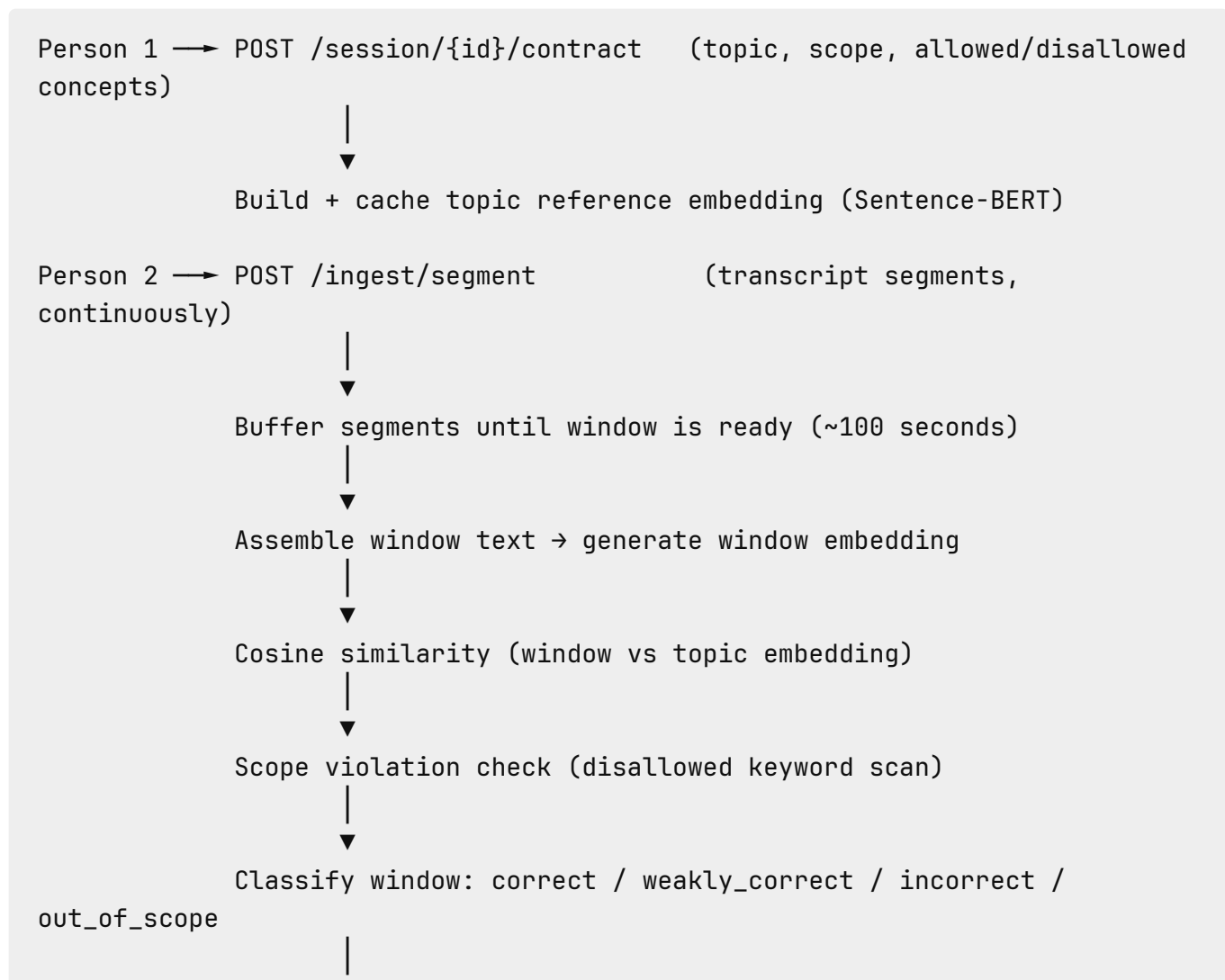
Semantic_Analysis

Semantic Analysis Engine (Sentence-BERT + Sliding Window)

Role

You are the brain of the session monitor. You receive a continuous stream of transcript segments from Person 2, assemble them into time windows, and classify each window against the agreed session topic using semantic similarity. Your classifications are the sole input Person 4 uses to decide whether to issue warnings. Accuracy of your embeddings and thresholds directly determines how well the system detects drift.

Data Flow



▼
POST result → Person 4 (port 8003)

What You Build

1. Service Setup – Port 8002

In-memory state (per active session):

- Session contract (topic, scope, allowed/disallowed concepts)
- Segment buffer (accumulates incoming segments)
- Window counter (tracks how many windows have been processed)
- Topic reference embedding (generated once when contract is registered, cached)

2. Contract Registration Endpoint

POST `/session/{barter_id}/contract`

Called by Person 1 when the session is created. Store the contract and immediately generate + cache the topic reference embedding.

Topic reference text = topic name + all allowed concepts joined as a single string.

Request:

```
{
  "barter_id": 1,
  "topic": "Python Basics",
  "scope": "beginner",
  "allowed_concepts": ["variables", "loops", "functions", "syntax",
    "lists"],
  "disallowed_concepts": ["decorators", "async", "metaclasses", "threading"]
}
```

Response:

```
{ "status": "contract registered", "barter_id": 1 }
```

3. Segment Ingest Endpoint

POST `/ingest/segment`

Called by Person 2 for every completed transcript segment. Append to buffer. When the buffer covers ~100 seconds of audio, freeze it as a window and process it.

Request:

```
{
  "barter_id": 1,
  "speaker": "user_1",
  "text": "Today we will cover basic addition and subtraction using whole numbers.",
  "timestamp_start": 1710234000.0,
  "timestamp_end": 1710234028.5
}
```

Response:

```
{ "status": "received" }
```

Window readiness check: `last_segment.timestamp_end - first_segment.timestamp_start ≥ 100 seconds`

4. Sliding Window Assembly

When a window fires:

1. Freeze the current buffer as this window's content
2. Clear the buffer for the next window
3. Combine all segment texts into one string
4. Remove filler words (um, uh, like, you know, basically, okay, right)
5. Generate embedding for the cleaned combined text
6. Run cosine similarity against the cached topic embedding
7. Run scope violation check
8. Classify the window
9. POST result to Person 4

Window size: **90–120 seconds** of audio content.

5. Similarity Scoring

Use **Sentence-BERT model** `all-MiniLM-L6-v2` (downloaded once, ~90MB, cached locally).

- Topic embedding: generated once from `topic + allowed_concepts` text at contract registration
 - Window embedding: generated fresh for each window from the cleaned combined text
 - Similarity: cosine similarity between the two vectors → score between 0.0 and 1.0
-

6. Scope Violation Check

Simple keyword scan – no embeddings needed. Check whether any disallowed concept string appears in the window text (case-insensitive). If any match found → `scope_violation: true`.

7. Window Classification

Condition	Classification
<code>scope_violation = true</code>	<code>out_of_scope</code>
<code>similarity ≥ 0.55</code>	<code>correct</code>
<code>0.35 ≤ similarity < 0.55</code>	<code>weakly_correct</code>
<code>similarity < 0.35</code>	<code>incorrect</code>

Thresholds (0.55 / 0.35) are starting values – calibrate with at least 5 manual test cases after setup.

8. Session End Endpoint

POST `/session/{barter_id}/end`

Called by Person 2 when the session ends. Flush any remaining buffered segments as a final partial window, process and forward it, then notify Person 3 the session is over. Clean up all in-memory state for that barter.

Output Format – Sent to Person 4

Every window result is POSTed to `http://localhost:8003/window/result:`

```
{
  "barter_id": 1,
  "window_id": 3,
  "classification": "incorrect",
  "similarity_score": 0.2341,
  "scope_violation": false,
  "timestamp_start": 1710234200.0,
  "timestamp_end": 1710234300.0,
  "text_preview": "So let me talk about how the stock market works and what
derivatives are..."
}
```

Field	Type	Description
barter_id	int	Which session
window_id	int	Sequential window number (1, 2, 3 ...)
classification	string	correct, weakly_correct, incorrect, or out_of_scope
similarity_score	float	Cosine similarity 0.0–1.0
scope_violation	bool	True if a disallowed concept was found
timestamp_start	float	Window start (Unix timestamp)
timestamp_end	float	Window end (Unix timestamp)
text_preview	string	First 200 chars of window text (for logging)

Interface Contracts

You Receive (inbound)

From	Endpoint	What
Person 1	POST /session/{barter_id}/contract	Topic, scope, concepts
Person 2	POST /ingest/segment	Transcript segments
Person 2	POST /session/{barter_id}/end	Session end signal

You Call (outbound)

Target	When	Endpoint
Person 4	After each window	POST http://localhost:8003/window/result
Person 4	After session end	POST http://localhost:8003/session/{barter_id}/end

What You Do NOT Build

- Audio capture or STT → Person 2

- Warning severity decisions or counters → Person 4
 - Warning messages shown to users → Person 4
 - Trust score updates → Person 1
 - QA verdict logic → Person 1
 - Frontend → Person 1
-

Libraries

fastapi, uvicorn, sentence-transformers, numpy, httpx, torch

Definition of Done

- ☐ Service runs on port 8002
- ☐ Contract registration generates and caches topic embedding correctly
- ☐ Segments buffer and windows fire at ~100s intervals
- ☐ Cosine similarity returns values in 0.0–1.0 range
- ☐ Scope violation check flags disallowed keywords correctly
- ☐ Window classification returns one of the 4 correct labels
- ☐ Every window result is POSTed to Person 4 within 2 seconds
- ☐ Session end flushes remaining segments and notifies Person 4
- ☐ In-memory state is cleaned up after session ends
- ☐ Thresholds documented with at least 5 manual test cases